Trees and Tries

Topic 3 of 10

30 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Find right-side view of binary tree

EASY | asked 2025-11-01 |

You are given the root of a binary tree. From the **right side of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**.**

```
def right_side_view(root):
```

2. Tree Node.

EASY | asked n/a |

Given a binary tree, identify which nodes are leaf nodes, root nodes or inner nodes. Leaf nodes: A node with no children Root nodes: A node with no parent Inner nodes: A node with both children and parent nodes You are given the following table tree_node_table: idp_idintegerinteger where p_id is the id of the parent node.

```
def make_db(rows):
```

3. Given two trees, determine if the pre-order traversals of two trees are same or not.

EASY | asked n/a |

Given two trees, determine if the pre-order traversals of two trees are same or not.

```
def same_preorder(a, b):
```

4. Calculate the height of a binary tree.

EASY | asked n/a |

Calculate the height of a binary tree.

```
def height(root):
```

5. Expression Tree Single-Leaf Mutation

MEDIUM | asked 2026-09-27 | tree, dfs, simulation, state-machine

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip.

```
def flip_answers(root):
```

6. Compute File System Total Size

MEDIUM | asked 2026-09-02 | tree, dfs

Examples: root = Dir(Dir(File(10), File(20)), File(5), Dir(File(30)),) Total: 65

```
def total_size(root, count_once=True):
```

7. Nested List Weight Sum (LC 339) with Follow-Ups

MEDIUM | asked 2026-08-16 | tree, recursion, parsing

Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3x1 + 8x1 + 2x2 + 14x2 + 2x2 + 91x3 = 320. Use [2, [91]] as a manual trace for bracket depth and the final-number flush.

```
def weight_sum(nested):
```

8. Longest Consecutive Path in Binary Tree

MEDIUM | asked 2026-07-02 | tree, dfs, recursion

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1.

```
def longest_consecutive(root, want_path=False):
```

9. Employee Hierarchy Add / Delete

MEDIUM | asked 2026-06-10 | tree, hashmap, object-design

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id.

```
class Org:

    def __init__(self, ceo):
```

```
    def add(self, emp, mgr):
```

```
    def senior(self, people):
```

```
    def delete(self, emp, policy='grandparent')
      :
```

```
    def reassign(self, emp, new_mgr):
```

```
    def move(self, emp, new_mgr):
```

10. Streaming Insert + Find K-th Largest

MEDIUM | asked 2026-02-04 | data-structure, tree

Examples: insert(3) insert(3) insert(2) findLargest(0) -> 3 findLargest(1) -> 3 findLargest(2) -> 2

```
class StreamKth:

    def __init__(self):
```

```
    def insert(self, x):
```

```
    def delete(self, x):
```

```
    def find_largest(self, k):
```

11. Find mode in a trinary search tree

MEDIUM | asked 2025-11-16 |

You are given the root of a **trinary search tree** where each node has up to three children: text class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all values > val } Assume the tree is intended to satisfy the ordering rules above. ## Task Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes. - If there is one mode, return a single-element list.

```
def find_modes(root):
```

12. Solve tree, graph, sliding-window problems

MEDIUM | asked 2025-08-04 |

Question LeetCode 103. Binary Tree Zigzag Level Order Traversal - given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule - given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters - given string s and integer k, return the length of the longest substring containing at most k distinct characters.

```
def zigzag(root):
```

13. Implement zigzag level-order traversal

MEDIUM | asked 2025-08-01 |

Given the root of a binary tree, return the node values in zigzag level order: visit nodes level by level, reading the first level from left to right, the next from right to left, and so on alternating each level. Output a list of lists of integers where each inner list contains the values for one level in the required direction. Aim for O(n) time and O(w) extra space, where n is the number of nodes and w is the maximum width of the tree.

```
def zigzag_level_order(root):
```

14. Construct a binary tree given its preorder and inorder traversals.

MEDIUM | asked n/a |

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node.

```
def build_pre_in(preorder, inorder):
```

15. Given the root of a binary tree, return the length of the diameter of the tree.

MEDIUM | asked n/a |

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree.

```
def diameter_nodes(root):
```

16. Determine if a given binary tree is a binary search tree (BST).

MEDIUM | asked n/a |

Determine if a given binary tree is a valid Binary Search Tree (BST).

```
def is_bst(root, dup='none'):
```

17. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

MEDIUM | asked n/a |

Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

```
def min_rounds(tree, root):
```

18. Lowest Common Ancestor of a Binary Tree III

MEDIUM | asked n/a |

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA). Each node has a parent pointer.

```
def lca_swap(p, q):
```

19. Lowest Common Ancestor

MEDIUM | asked n/a |

Given a binary tree and two nodes p and q, find their lowest common ancestor (LCA): the lowest node that has both p and q as descendants (a node can be a descendant of itself).

```
def lca(root, p, q):
```

20. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

HARD | asked 2026-09-16 |

Given a binary tree and an integer target, delete every leaf whose value equals target. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals target, it must be deleted as well, and so on, until no leaf with value target remains. Return the resulting tree, which is empty if the root itself ends up deleted.

```
def remove_leaves(root, target):
```

21. Huffman Tree Construction

HARD | asked 2026-06-01 | tree, heap, greedy

Examples: {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45} -> standard Huffman tree with f near the root.

```
def huffman_codes(freq):
```

22. Compute distance-sum from every tree node

HARD | asked 2026-01-22 |

Overview: This question evaluates understanding of tree and graph algorithms, including computation of pairwise distances and techniques for aggregating path lengths across nodes.

```
def distance_sums(n, edges):
```

23. Implement a decode and encode function of a general N-ary tree.

HARD | asked n/a |

Implement a decode and encode function of a general N-ary tree.

```
def encode(root):
```

24. URL Request Handler Router

MEDIUM | asked 2026-10-16 | string, parsing, data-structure, trie, dfs

Examples: Register /users/{userId}/photo/{photoId}. A lookup for /users/123/photo/456 matches that registered handler.

```
class Router:

    def __init__(self):
```

```
    def add(self, pattern, handler=None, method
      ='GET'):
```

```
    def match(self, url, method='GET'):
```

25. Trie Prefix Search / Autocomplete

MEDIUM | asked 2026-07-02 | trie, tree, dfs

LeetCode 208 + 1268. Build a trie from a dictionary and return all words sharing a query prefix. Standard Google onsite coding. Build a trie from a dictionary / array of words. Support searchPrefix(prefix) -> list[str]: return all words that start with the query prefix, lexicographically sorted unless ranking is specified.

```
class Trie:

    def __init__(self, k=3):
```

```
    def insert(self, word, freq=1):
```

```
    def find(self, prefix):
```

```
    def search_prefix(self, prefix):
```

```
    def suggest(self, prefix):
```

```
    def fuzzy(self, prefix):
```

```
    def search_below(self, node, text):
```

26. Greedy Longest-Match Dictionary Tokenizer

MEDIUM | asked 2026-06-15 | string, trie, hashmap, greedy

Phone-screen coding round (R1). Input: a text string and a dictionary mapping key (string) -> id. Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one. Output the concatenation (or list) of ids and literal characters in order. Adapt the data structure so the rule swap is cheap.

```
class Tokenizer:

    def __init__(self, table, priority=None):
```

```
    def better(self, new, old):
```

```
    def longest_at(self, text, i):
```

```
    def tokenize(self, text):
```

27. Check if all substrings are dictionary words

MEDIUM | asked 2026-03-11 |

You are given a string s (letters only) and access to an English dictionary dict (a set of valid words). Return true if **every contiguous substring of s with length **>= 3** is a valid dictionary word (i.e., for all i, j with 0 <= i <= j < n and j - i + 1 >= 3, s[i..j] in dict). Otherwise return false. Clarifications: - Substring means **contiguous**. - You may assume dictionary lookups are available (e.g., dict.contains(word)), or you can preprocess the dictionary.**

```
def all_substrings_words(s, words, min_len=3):
```

28. Return dictionary words matching a prefix

MEDIUM | asked 2026-03-09 |

You are given a list of strings words (a dictionary) and a string prefix. Return all words in words that start with prefix. Clarifications/constraints: - Matching is case-sensitive. - If no words match, return an empty list. - If words contains duplicates, include duplicates in the output.

```
class PrefixIndex:

    def __init__(self, words):
```

```
    def matches(self, prefix, limit=None):
```

```
    def add(self, word):
```

29. Solve chat, grid paths, and car rentals

MEDIUM | asked 2025-12-02 |

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. --- ### Q1. Find top-K most active chat users You are given a log of chat messages from a messaging application. **Input**: - An array messages of length n, where each element messages[i] is the user_id (string or integer) of the sender of the i-th message. - An integer k (1 <= k <= number of distinct users).

```
def top_k_users(messages, k):
```

30. Implement Longest-Match Text Replacement

HARD | asked 2026-01-20 |

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form token:id, where token is a string pattern and id is the value that should be emitted when that token is matched. At the current text position, find all dictionary tokens that match the substring starting at that position. 2. If at least one token matches, choose the longest matching token, append its corresponding id to the output, and advance by the length of that token. 3.

```
class Replacer:

    def __init__(self, mapping):
```

```
    def candidates(self, text, i):
```

```
    def replace(self, text, choose=None, word_b
      oundary=False):
```

```
    def edge(text, i, size):
```